TIVIDY

Workflow System

COS 214

Project

The Team

Tamlyn Khan (u24675815)

Yoshua Smit (u25433726)

Isabelle Engelbrecht (u25090501)

Jonathan Witter (u25108582)

Sudud Abughazaleh (u25282728)

PDF Content

1. **Team Contribution Statement**.....4

2. **GitHub Link**.....4

3. **Workflow Research**.....5
    a. References
    b. Assumptions
    c. Boundaries

4. **Industries**.....11
    a. Education Industry
    b. Game Industry

5. **UML and Other Diagrams**.....17
    a. UML Class Diagram
    b. Sequence Diagrams
    c. Activity Diagrams

6. **GoF Participant Mapping and Pattern Justification**....24

7. **Implementation Explanation**.....41

8. **Design Decision History**.....42

9. **Engineering Evidence**.....49
    a. Docker
    b. Doxygen
    c. Testing
    d. GitHub contribution
        - Branches
        - Pull requests, Merges and Commits

Team contribution statement

- **Tamlyn** – Research, three activity diagrams, three GoF patterns(State, Façade, Iterator), Docker, GUI and the main.cpp.
- **Yoshua** - Design pattern justification, testing, Doxygen, one GoF pattern(Mediator).
- **Bella** - scenario UML class diagrams, design pattern justifications and participant mapping, PDF, Slides and three GoF patterns(Factory Method, Decorator, Observer).
- **Jonathan** - Generic UML class diagram and sequence diagram, templatized the generic system and three GoF patterns(Prototype,Composite,Adapter).
- **Sudud** - sequence diagram, state diagram, PDF, Slides and three GoF patterns(Chain of Responsibility,Strategy,Memento).

GitHub

https://github.com/IridescentDucky/COS214-Practical-6-Project.git

Workflow Management Research

- **What is a workflow and what is a workflow management system?**

A workflow is an automated process where tasks pass between people according to set rules. A WfMS stores, runs and manages these processes (Hollingsworth, 1995; WfMC, 1996). So TIVIDY's engine must run processes without containing them.

- **What is the difference between a workflow definition and a running workflow instance?**

A definition is the template of steps, order and roles; an instance is one run of it, with its own data and history (WfMC, 1996). So these need separate classes, and changing one instance must not affect the others.

- **How are activities or work items represented?** Activities create work items that go to a worklist and move through states such as waiting, active and completed (WfMC, 1996). So work items need a state-based lifecycle.

- **How are work items assigned and routed?** Routing uses sequence, parallel, choice and loops, and each product supports a different subset (van der Aalst et al., 2003). Work is either pushed to someone by role or workload, or

pulled from a shared list (Russell et al., 2005). So both routing and assignment must be swappable, not hard-coded.

- **How are approvals, rejection and escalation handled?**

An approval is a work item whose result decides the next step. Rejection sends work back, and escalation reassigns it when a deadline passes (WfMC, 1996; Russell et al., 2005). So deadlines and escalation rules must be configurable.

- **How are participants, roles and responsibilities represented?**

Definitions name roles, not people, and the engine picks a person at run time from the organisation's structure (WfMC, 1996). So staff changes should not require changing definitions.

- **How are dependencies, events and failures handled?**

Steps wait on conditions or joins, events trigger actions, and each type of failure gets a chosen response such as retry, skip or undo (WfMC, 1996; Russell, van der Aalst and ter Hofstede, 2006). So many parts must be able to react to one event, and failure handling must be configurable.

- **How do workflow systems interact with external systems?**

Engines reach outside systems through a separate interface (Hollingsworth, 1995); Camunda, for example, queues jobs for external workers to pick up (Camunda, n.d.b). So TIVIDY needs a translation layer for each external system.

- **How is workflow history or audit information recorded?**

Every state change is logged with a timestamp (WfMC, 1996). So actions should be stored as history objects, which also makes undo possible.

- **How are running workflows monitored and reported on?**

Monitoring is separate from execution, and logs are analysed afterwards to find bottlenecks (Hollingsworth, 1995; van der Aalst, 2016). So reporting should read the history from outside the core classes.

- **What makes a workflow engine reusable across different processes?**

All process rules live in the definition, not the engine (WfMC, 1996). So no scenario specific code belongs in the engine.

• **Which information belongs to a workflow definition and which belongs to a running instance?** Definition: steps, order, conditions, roles and deadlines. Instance: current state, data, assigned people and history (WfMC, 1996).

• **How do real workflow systems model nested activities or sub-processes?**

A sub-process has its own definition and can be nested inside others (WfMC, 1996). So nested work forms a tree the engine can traverse.

• **Which decisions are normally configurable rather than hard-coded into the workflow engine?** Routing, assignment, escalation and failure handling (WfMC, 1996; Russell et al., 2005; Russell, van der Aalst and ter Hofstede, 2006).

• **How are changes to workflow definitions handled while work is already in progress?**

A change creates a new version, and running instances stay on the old one unless they can be safely migrated (Camunda, n.d.a; Rinderle, Reichert and Dadam, 2004). So definitions must be versioned.

References

- Camunda (n.d.a) Process versioning. Available at: https://docs.camunda.org/manual/latest/user-guide/process-engine/process-versioning

  (Accessed: 1 October 2026).

- Camunda (n.d.b) External tasks. Available at: https://docs.camunda.org/manual/latest/user-guide/process-engine/external-tasks

  (Accessed: 1 October 2026).

- Hollingsworth, D. (1995) The workflow reference model. Brussels: Workflow Management Coalition.
- Rinderle, S., Reichert, M. and Dadam, P. (2004) 'Correctness criteria for dynamic changes in workflow systems: a survey' , Data & Knowledge Engineering, 50(1), pp. 9–34.
- Russell, N., van der Aalst, W.M.P. and ter Hofstede, A.H.M. (2006) 'Workflow exception patterns', in CAiSE 2006. Berlin: Springer, pp. 288–302.
- Russell, N., van der Aalst, W.M.P., ter Hofstede, A.H.M. and Edmond, D. (2005) 'Workflow resource patterns' , in CAiSE 2005. Berlin: Springer, pp. 216–232.
- van der Aalst, W.M.P. (2016) Process mining: data science in action. 2nd edn. Berlin: Springer.
- van der Aalst, W.M.P., ter Hofstede, A.H.M., Kiepuszewski, B. and Barros, A.P. (2003) 'Workflow patterns' , Distributed and Parallel Databases, 14(1), pp. 5–51.
- Workflow Management Coalition (WfMC) (1996) Terminology & glossary. Brussels: Workflow Management Coalition.

Assumptions

We assume that the tasks and state transitions follow a logical sequence. Thus, some actions happen first before others. Example : The Factory Method creates the WorkItem product **first** and then the Prototype clones the WorkItem.

Boundaries

We can only have a set number of **types** of conditions including <, >, ==, <=,>= for the Concrete Handlers in the Chain of Responsibilities.

Specific systems are bounded by the **number** of States that the generic system has, for example Created, Delivered, inProgress.

Any specific system that implements the generic system **decorators** are bounded by ExtraTime and BonusScore functionality.

The system is bounded by the **limited number** of Gang of Four patterns: Façade, Factory Method, Mediator, Strategy, Prototype, Adapter, Chain of Responsibility, Observer, Composite, Decorator, State, Memento.

Industries

Education Industry (Exam Workflow)

Why the Exam Scenario fits the generic system:

Marking an exam paper involves a series of tasks that must be completed in a particular order by different participants. The system needs to track the lifecycle of the exam paper(inProgress,Reviewed,etc.). The exam paper can be created and cloned and statistics from the paper can be saved and stored. The paper may also need extra functionality like bonus marks or extra time. The generic system accommodates these requirements by providing a structure for managing tasks, tracking their states, cloning items as well as creating them and the generic system also provides a way to store a snapshot of statistics. The generic system also has a way to provide additional functionality.

Processes :

The education workflow focuses on the process of marking and capturing marks for examination papers.

An exam paper is created, cloned, then after writing the exam papers will be marked and lastly the marks for the exam paper will be captured and saved.

Participants :
The participants in this workflow come from different GoF patterns which are picked from the Generic Workflow System.

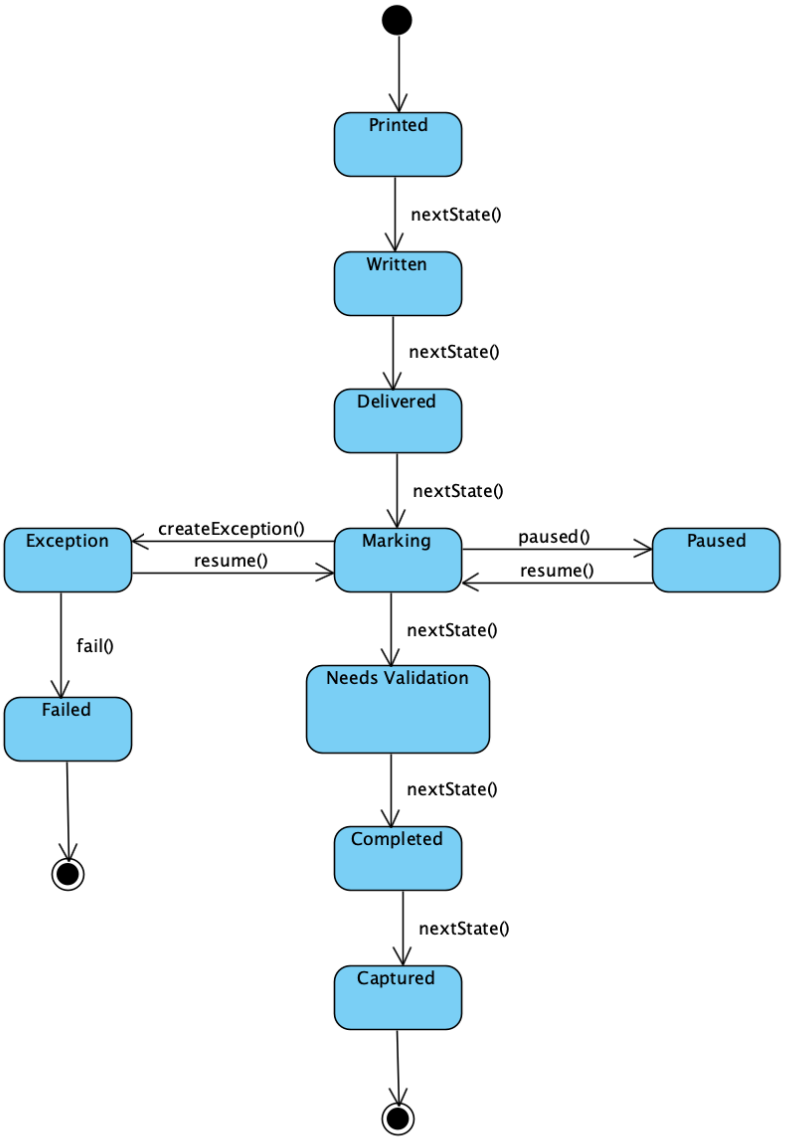
Here patterns like Iterator, Prototype, Memento, Strategy, Chain of Responsibility, Observer, State, Façade, Factory Method and Decorator are used from the generic system to build a specific system.

Branching :
The workflow can branch depending on the marking results and whether a paper requires further review.

For example, a paper that does not require further escalation can continue through the normal marking process, while a paper with discrepancies, physical errors or other issues can be forwarded to the Lecturer for further review. This allows different examination papers to follow different routes depending on their circumstances.

Failure paths and outcomes :
The possible failure paths and outcomes are represented through the state diagrams. These illustrate what happens when marking cannot continue as expected, such as when invalid marks are entered or an issue requires further intervention. The successful outcome is that the examination paper completes the marking process and receives a final recorded mark.

State diagram to show failure paths and outcomes for the Exam scenario :



Gaming Industry (Game Workflow)

Why the Game Scenario fits the generic system:

The game development workflow fits the generic system because developing a game also involves multiple tasks, stages and participants that need to be coordinated. A game progresses through stages such as concepting, writing, asset building, coding, testing and release, with different workers responsible for completing each stage. The game's lifecycle also needs to be tracked and when the state changes different departments must be notified. We should also be able to save the state of the game so that the state can be restored if needed.

The generic system can manage these activities, track the game's progress and allow the lifecycle to be tracked. The generic system also provides a way to inform/notify the handlers of a state change. There is also a way to store a snapshot of the item.

Processes :

The gaming workflow focuses on the development of a game, from its initial concept through to its release.

The process begins with the concepting stage, followed by writing, asset building and coding where tests are run and then lastly after validation the game gets released.

Participants :

The participants in this workflow come from different GoF patterns which are picked from the Generic Workflow System.

Here patterns like Façade, State, Composite, Prototype, Memento, Strategy, Chain of Responsibility, Observer, Factory Method and Decorator are used to implement the system.

Branching :

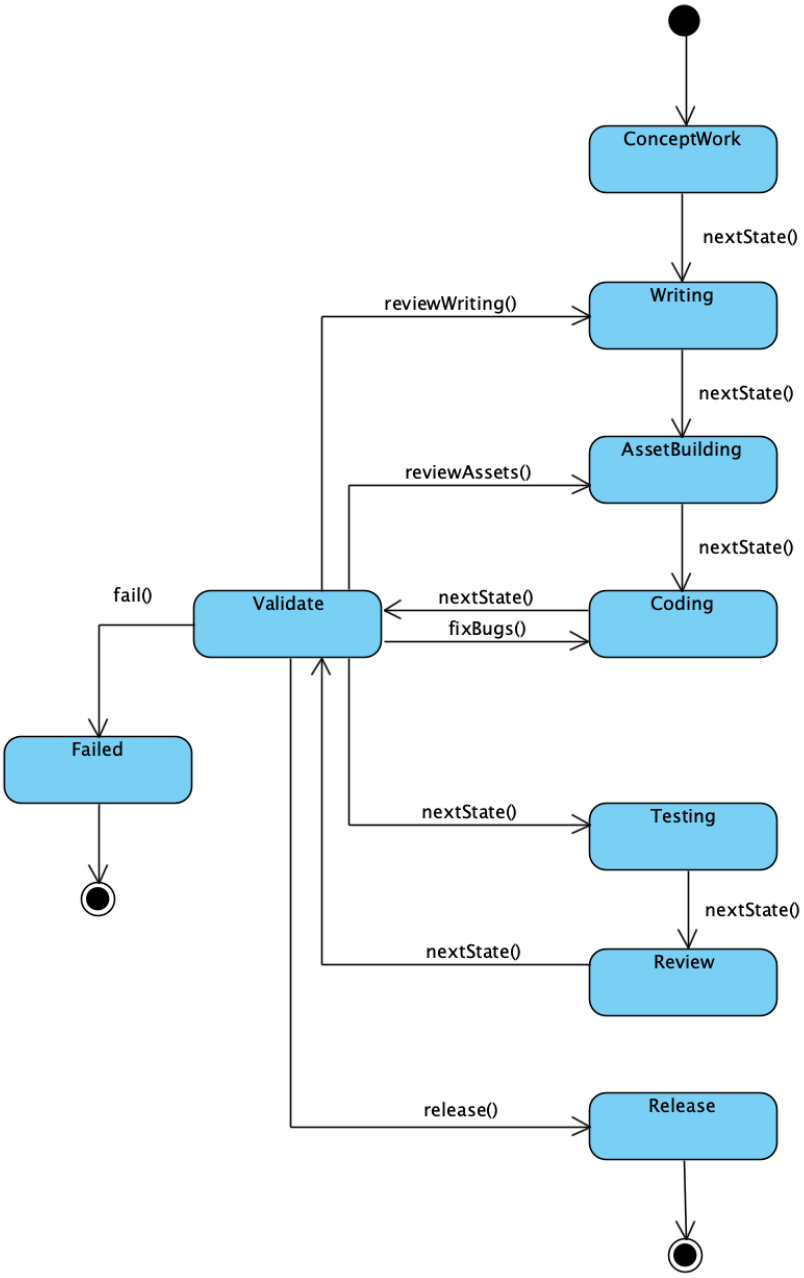
The game development workflow can follow different paths depending on the results of validation and review. For example, if a game passes validation, it can proceed to testing or release. However, if bugs are identified, it can return to coding for corrections.

Feedback can require the workflow to return to writing or asset building so that changes can be made before development continues.

Failure paths and outcomes :

The state diagrams show the possible failure paths and outcomes of the game development process. These include returning to earlier development stages when bugs are found or when work does not meet the required standards. The successful outcome is that the game completes the required development and testing stages and reaches the release stage.

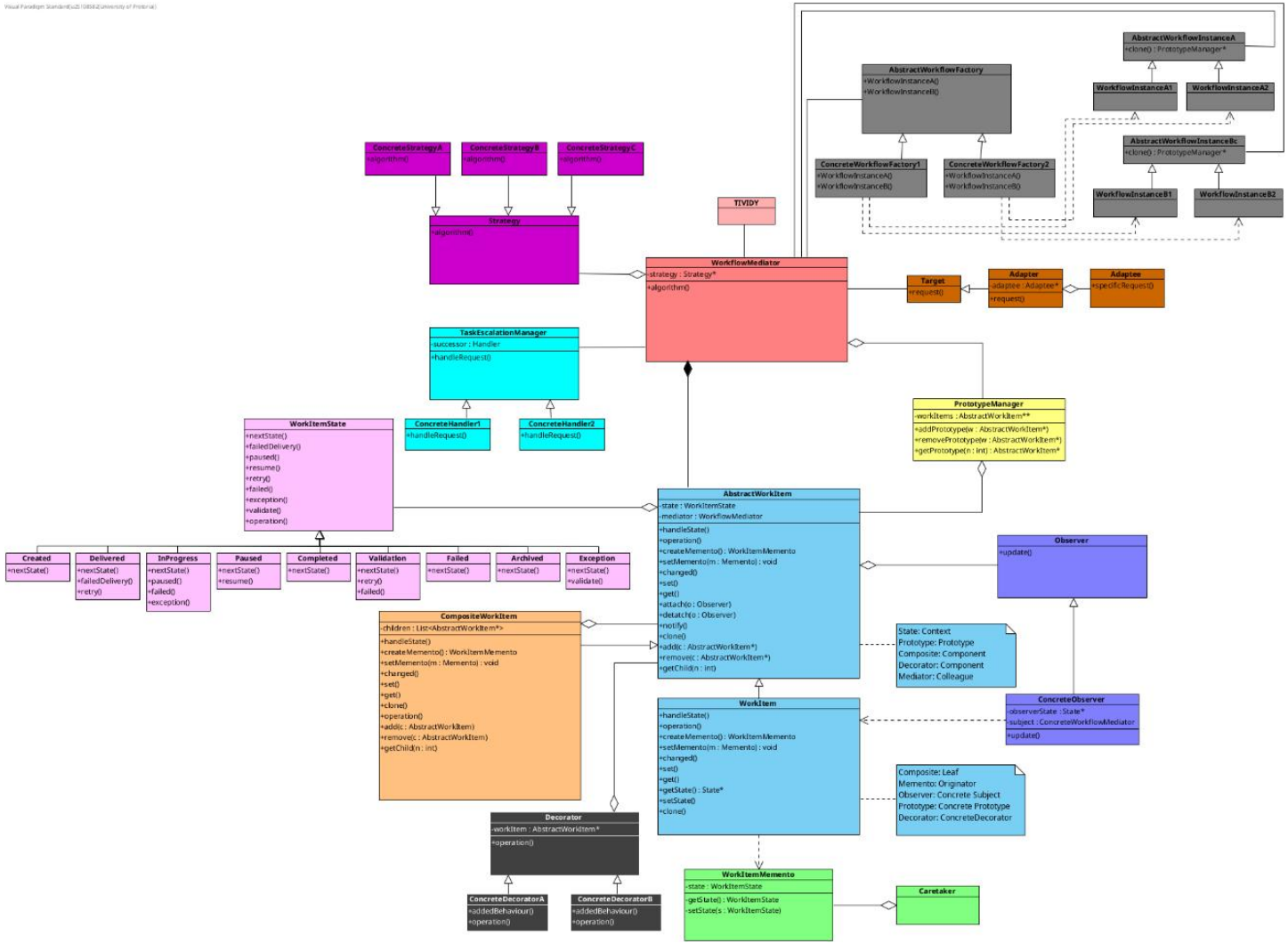
State diagram to show failure paths and outcomes for the Exam scenario:



UML and Other Diagrams

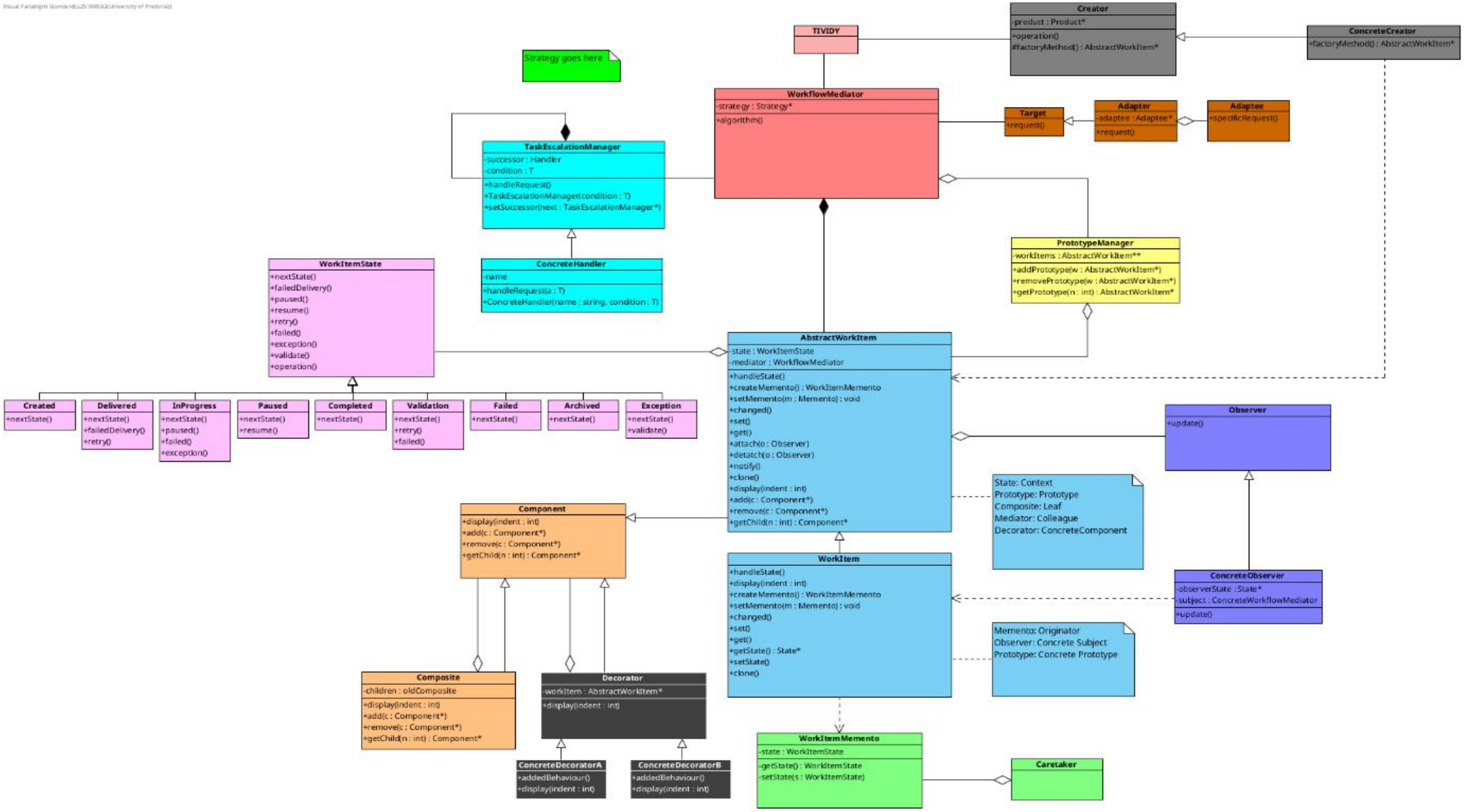
Generic UML

Different versions of our Generic UML in **chronological order** :



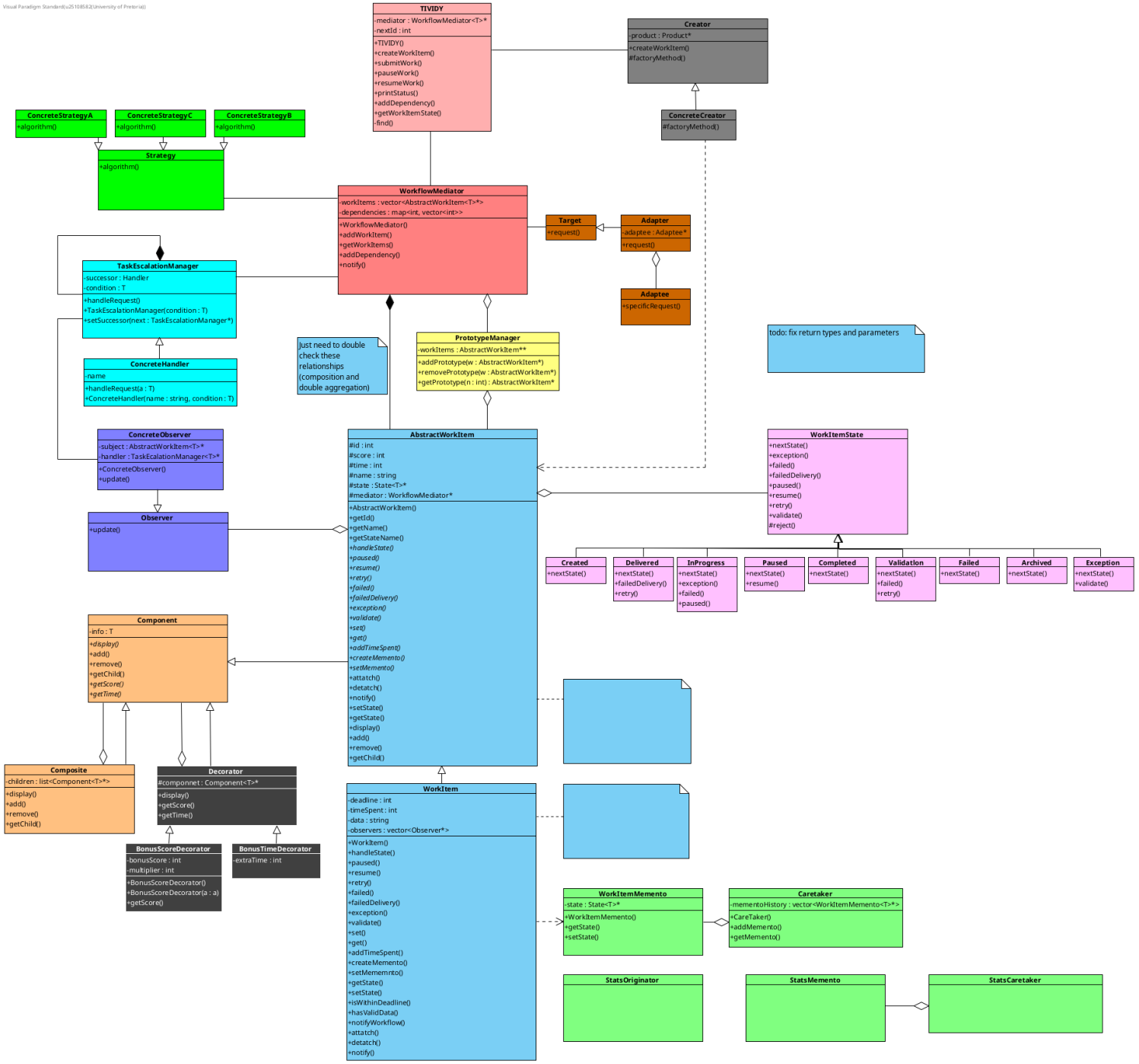
Some changes made :

- Changed Abstract Factory to Factory method – We are not creating families of products, we are only producing one WorkItem product thus we changed the pattern.
- Changed the Component participant(Composite pattern – Orange) to have a separate Component class instead of using the AbstractWorkItem as the component – separating the classes separates the responsibilities and enables the Decorator pattern to use the Component as well.


Visual Paradigm Standard(u2570882@University of Pretoria))
Strategy goes here
TIVIDY
Creator
-product : Product*
+operation()
#factoryMethod() : AbstractWorkItem*
ConcreteCreator
+factoryMethod() : AbstractWorkItem*
WorkflowMediator
-strategy : Strategy*
+algorithm()
Target
+request()
Adapter
-adaptee : Adaptee*
+request()
Adaptee
+specificRequest()
TaskEscalationManager
-successor : Handler
-condition : T
+handleRequest()
+TaskEscalationManager(condition : T)
+setSuccessor(next : TaskEscalationManager*)
ConcreteHandler
-name
+handleRequest(a : T)
+ConcreteHandler(name : string, condition : T)
PrototypeManager
-workItems : AbstractWorkItem**
+addPrototype(w : AbstractWorkItem*)
+removePrototype(w : AbstractWorkItem*)
+getPrototype(n : int) : AbstractWorkItem*
WorkItemState
+nextState()
+failedDelivery()
+paused()
+resume()
+retry()
+failed()
+exception()
+validate()
+operation()
AbstractWorkItem
-state : WorkItemState
-mediator : WorkflowMediator
+handleState()
+createMemento() : WorkItemMemento
+setMemento(m : Memento) : void
+changed()
+set()
+get()
+attach(o : Observer)
+detatch(o : Observer)
+notify()
+clone()
+display(indent : int)
+add(c : Component*)
+remove(c : Component*)
+getChild(n : int) : Component*
Created
+nextState()
Delivered
+nextState()
+failedDelivery()
+retry()
InProgress
+nextState()
+paused()
+failed()
+exception()
Paused
+nextState()
+resume()
Completed
+nextState()
Validation
+nextState()
+retry()
+failed()
Failed
+nextState()
Archived
+nextState()
Exception
+nextState()
+validate()
Observer
+update()
State: Context
Prototype: Prototype
Composite: Leaf
Mediator: Colleague
Decorator: ConcreteComponent
Component
+display(indent : int)
+add(c : Component*)
+remove(c : Component*)
+getChild(n : int) : Component*
WorkItem
+handleState()
+display(indent : int)
+createMemento() : WorkItemMemento
+setMemento(m : Memento) : void
+changed()
+set()
+get()
+getState() : State*
+setState()
+clone()
ConcreteObserver
-observerState : State*
-subject : ConcreteWorkflowMediator
+update()
Memento: Originator
Observer: Concrete Subject
Prototype: Concrete Prototype
Composite
-children : oldComposite
+display(indent : int)
+add(c : Component*)
+remove(c : Component*)
+getChild(n : int) : Component*
Decorator
-workItem : AbstractWorkItem*
+display(indent : int)
WorkItemMemento
-state : WorkItemState
-getState() : WorkItemState
-setState(s : WorkItemState)
Caretaker
ConcreteDecoratorA
+addedBehaviour()
+display(indent : int)
ConcreteDecoratorB
+addedBehaviour()
+display(indent : int)

Some changes made :

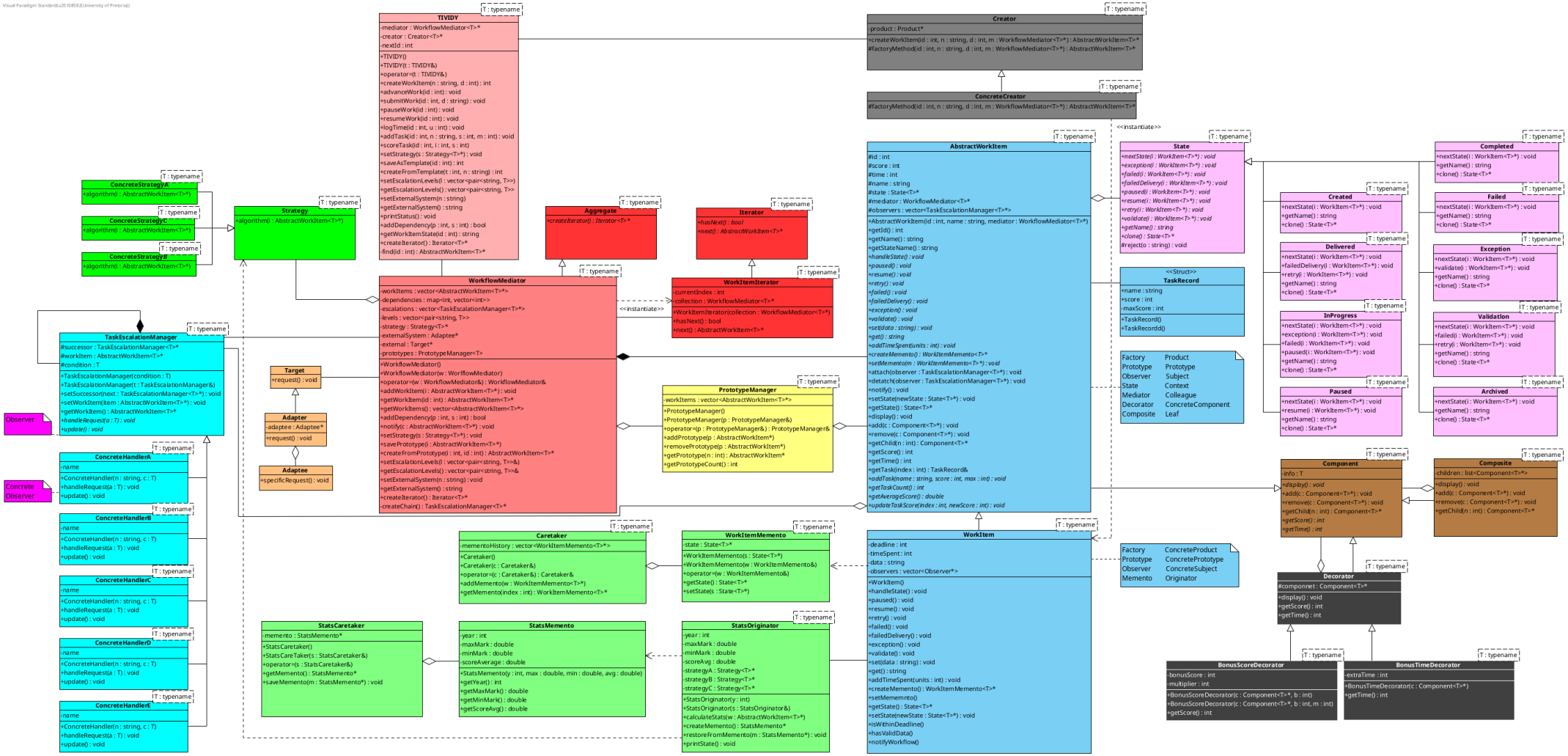
- Added the strategy pattern – We use strategy to calculate statistics like the MAX and MIN.
- Added functions to the Façade class(TIVIDY Class) – The Client can use the functions in the Façade instead of having to use the functions from the patterns directly, meaning we promote loose coupling in the system.
- Added an additional Memento pattern – We use two different mementos because they each have a different use case in our system. One Memento takes a snapshot of the entire WorkItem so that the WorkItem may be restored to the state at a later time. The other memento takes a snapshot of the statistics which we calculate using Strategy. Because of the two different applications we decided to include two memento use cases.
- Changed the decorator pattern to have extraTime and bonusScore – we needed generic additional functionality options that can be added to a specific system.


Visual Paradigm Standard(u2510858(University of Pretoria))
TIVIDY
-mediator : WorkflowMediator<T>*
-nextId : int
+TIVIDY()
+createWorkItem()
+submitWork()
+pauseWork()
+resumeWork()
+printStatus()
+addDependency()
+getWorkItemState()
-find()
Creator
-product : Product*
+createWorkItem()
#factoryMethod()
ConcreteCreator
#factoryMethod()
ConcreteStrategyA
+algorithm()
ConcreteStrategyC
+algorithm()
ConcreteStrategyB
+algorithm()
Strategy
+algorithm()
WorkflowMediator
-workItems : vector<AbstractWorkItem<T>*>
-dependencies : map<int, vector<int>>
+WorkflowMediator()
+addWorkItem()
+getWorkItems()
+addDependency()
+notify()
Target
+request()
Adapter
-adaptee : Adaptee*
+request()
Adaptee
+specificRequest()
TaskEscalationManager
-successor : Handler
-condition : T
+handleRequest()
+TaskEscalationManager(condition : T)
+setSuccessor(next : TaskEscalationManager*)
ConcreteHandler
-name
+handleRequest(a : T)
+ConcreteHandler(name : string, condition : T)
Just need to double check these relationships (composition and double aggregation)
PrototypeManager
-workItems : AbstractWorkItem**
+addPrototype(w : AbstractWorkItem*)
+removePrototype(w : AbstractWorkItem*)
+getPrototype(n : int) : AbstractWorkItem*
todo: fix return types and parameters
ConcreteObserver
-subject : AbstractWorkItem<T>*
-handler : TaskEscalationManager<T>*
+ConcreteObserver()
+update()
Observer
+update()
AbstractWorkItem
#id : int
#score : int
#time : int
#name : string
#state : State<T>*
#mediator : WorkflowMediator*
+AbstractWorkItem()
+getId()
+getName()
+getStateName()
+handleState()
+paused()
+resume()
+retry()
+failed()
+failedDelivery()
+exception()
+validate()
+set()
+get()
+addTimeSpent()
+createMemento()
+setMemento()
+attatch()
+detatch()
+notify()
+setState()
+getState()
+display()
+add()
+remove()
+getChild()
WorkItemState
+nextState()
+exception()
+failed()
+failedDelivery()
+paused()
+resume()
+retry()
+validate()
#reject()
Created
+nextState()
Delivered
+nextState()
+failedDelivery()
+retry()
InProgress
+nextState()
+exception()
+failed()
+paused()
Paused
+nextState()
+resume()
Completed
+nextState()
Validation
+nextState()
+failed()
+retry()
Failed
+nextState()
Archived
+nextState()
Exception
+nextState()
+validate()
Component
-info : T
+display()
+add()
+remove()
+getChild()
+getScore()
+getTime()
Composite
-children : list<Component<T>*>
+display()
+add()
+remove()
+getChild()
Decorator
#componnet : Component<T>*
+display()
+getScore()
+getTime()
BonusScoreDecorator
-bonusScore : int
-multiplier : int
+BonusScoreDecorator()
+BonusScoreDecorator(a : a)
+getScore()
BonusTimeDecorator
-extraTime : int
WorkItem
-deadline : int
-timeSpent : int
-data : string
-observers : vector<Observer*>
+WorkItem()
+handleState()
+paused()
+resume()
+retry()
+failed()
+failedDelivery()
+exception()
+validate()
+set()
+get()
+addTimeSpent()
+createMemento()
+setMememnto()
+getState()
+setState()
+isWithinDeadline()
+hasValidData()
+notifyWorkflow()
+attatch()
+detatch()
+notify()
WorkItemMemento
-state : State<T>*
+WorkItemMemento()
+getState()
+setState()
Caretaker
-mementoHistory : vector<WorkItemMemento<T>*>
+CareTaker()
+addMemento()
+getMemento()
StatsOriginator
StatsMemento
StatsCaretaker

Some changes made :

- Filled in empty notes with meaningful struct declaration as well as information regarding patterns.
- Added template declarations for each template class.
- Added more concreteHandlers to expand the Chain of Responsibility – the generic should expand options to allow the accommodation of multiple handlers in scenarios.
- Removed the Observer and ConcreteObserver classes since the Chain of Responsibility(TaskEscalationManager) are the actual observers – when the WorkItem changes state the observers(Chain of Responsibility) must be notified, and the appropriate ConcreteHandler will handle the state change, thus we gave TaskEscalationManager the responsibility of observing WorkItem.
- Added/removed necessary functions for each design pattern – ex. WorkItem is the ConcreteSubject in the Observer pattern and thus should not have detach/attach functions since the Subject(AbstractWorkItem) handles it.
- Cleaned up the structure.

Final UML Class Diagram(Generic



GoF participant mapping and Pattern Justification

Memento

(WorkItem State)

Design problem in TIVIDY:

The system must save and restore a WorkItem's prior lifecycle state and progress without exposing its internal representation to the object that stores the snapshot.

GoF participant mapping:

- Originator: WorkItem

- Memento: WorkItemMemento

- Caretaker: Caretaker

How participants collaborate:

WorkItem creates a WorkItemMemento through createMemento() and restores it through setMemento(). Caretaker stores the memento but does not inspect its internals. In the EMS scenario, ExamStatisticsOriginator, StatisticsMemento, and StatisticsCaretaker apply the same collaboration to statistics snapshots.

What may vary:

- The state captured in a snapshot

- When snapshots are taken and restored

Why this pattern is appropriate:

TIVIDY can support pause, rollback, or recovery without breaking WorkItem encapsulation.

Memento

(Statistics)

Design problem in TIVIDY:

The system must save and restore a WorkItem's calculated performance statistics—generated dynamically by the Strategy pattern—without exposing their internal data structures to the object keeping track of historical records.

GoF participant mapping:

- Originator: StatsOriginator

- Memento: StatsMemento

- Caretaker: StatsCaretaker

How participants collaborate:

StatsOriginator creates a StatsMemento to snapshot the calculated statistics (such as min and max) and restores them when needed. StatsCaretaker holds onto these

snapshots without altering or inspecting their internal contents.

What may vary:

- The specific calculated statistics captured in the snapshot (like score calculations or performance averages)

Why this pattern is appropriate:

TIVIDY can track and recover historical performance data independently from the workflow item's core state, keeping the calculation logic clean and fully encapsulated.

Factory Method and Prototype

Design problem in TIVIDY:

The system needs to create new workflow items without hard-coding their specific classes, while also allowing existing workflow templates to be reproduced seamlessly through cloning.

GoF participant mapping:

- Creator: Creator
- Concrete Creator: ConcreteCreator
- Product: AbstractWorkItem
- Concrete Product: WorkItem
- Prototype: AbstractWorkItem
- Concrete Prototype: WorkItem

How participants collaborate:

Clients invoke the factoryMethod() on a ConcreteCreator to instantiate new work items dynamically without coupling the code to specific subclasses. Concurrently, the PrototypeManager stores established workflow item templates so they can be cloned to quickly create independent, running instances.

What may vary:

- The concrete subclass of work items instantiated by the factory.

- The specific workflow template chosen from the prototype manager for duplication.

Why these patterns are appropriate:

The Factory Method keeps our code flexible by handling work item creation behind the scenes, making it easy to introduce new types down the road. The Prototype pattern clones existing templates instead of building everything from scratch.

Strategy

Design problem in TIVIDY:

WorkItems require performance statistics to be computed, but these calculations shouldn't be hardcoded

GoF participant mapping:

- Context: WorkflowMediator

- Strategy: Strategy

- Concrete Strategies: ConcreteStrategyA, ConcreteStrategyB, ConcreteStrategyC

How participants collaborate:

WorkflowMediator holds a Strategy reference and delegates algorithm() to the selected concrete strategy. The strategy supplies the decision while the mediator remains responsible for workflow coordination

What may vary:

- The statistical algorithm used to calculate specific stats (like min, max, or averages) from WorkItem.

Why this pattern is appropriate:

Strategy decouples statistical calculation logic from the WorkItem itself. With Strategy we can add new calculation algorithms without changing the underlying structure.

State

Design problem in TIVIDY:

A WorkItem behaves differently as it progresses through its lifecycle. Operations valid while work is in progress must not be accepted after it has completed or been archived.

GoF participant mapping:

- Context: AbstractWorkItem

- State: State

- Concrete states: Created, Delivered, InProgress, Paused, Completed, Failed, Exception, Validation, Archived

How participants collaborate:

AbstractWorkItem(Context) stores its current State and uses transition functions like handleState(), retry(), failed(),etc. to change its state.

What may vary:

- Legal transitions and state-specific behaviour.

Why this pattern is appropriate:

State prevents lifecycle checks from becoming a large conditional chain in WorkItem and keeps each state's transition rules together.

Composite

Design problem in TIVIDY:

Workflows can contain both individual work items and nested groups of work. Clients should be able to handle both through one interface.

GoF participant mapping:

- Component: Component

- Composite: Composite

- Leaf: AbstractWorkItem

How participants collaborate:

Composite provides add(), remove(), getChild(). AbstractWorkItem is the leaf. We can build groups containing WorkItems or Composite nodes.

What may vary:

- The nesting and grouping of work items

- The number and structure of child workItems

Why this pattern is appropriate:

It models stages and sub-processes while preserving a uniform interface for workflow traversal and execution.

Decorator

Design problem in TIVIDY:

A work item may need optional responsibilities without creating a subclass for every combination of behaviours.

GoF participant mapping:

- Component: AbstractWorkItem

- Decorator: Decorator

- Concrete decorators: BonusScoreDecorator, BonusTimeDecorator

How participants collaborate:

Decorator wraps the WorkItem with additional functionality by adding extra time or by altering the score. These decorators can also be stacked.

What may vary:

- The combination and order of added responsibilities

Why this pattern is appropriate:

Decorator layers independent concerns onto a task at runtime while retaining the same component interface.

Observer

Design problem in TIVIDY:

Several components may need to react to one workflow event without the workflow coordinator knowing the details of every reaction.

GoF participant mapping:

- Subject: AbstractWorkItem

- Concrete subject: WorkItem

- Observer: TaskEscalationManager

- Concrete observer: ConcreteHandlerA, ConcreteHandlerB,ConcreteHandlerC, ConcreteHandlerD, ConcreteHandlerE

How participants collaborate:

When the WorkItem changes state the observers(Chain of Responsibility) will be notified. The Observer(TaskEscalationManager) will send a new request through the chain and the appropriate ConcreteHandler from the chain will handle the updated state.

What may vary:

- The observers registered

Why this pattern is appropriate:

Notifications, monitoring, and downstream reactions should be independently extendable rather than hard-coded into the coordinator or workItem.

Adapter

Design problem in TIVIDY:

The generic workflow engine must use external services without depending on their incompatible interfaces.

GoF participant mapping:

- Client: WorkflowMediator

- Target: Target

- Adapter: Adapter

- Adaptee: Adaptee

How participants collaborate:

WorkflowMediator depends on Target and calls request(). Adapter implements Target, holds an Adaptee, and translates request() into the adaptee's specificRequest().

What may vary:

- The external service or legacy interface

- The translation between a workflow operation and an external API

Why this pattern is appropriate:

Adapter keeps TIVIDY independent of a particular external system while allowing a suitable connector to be introduced or replaced.

Mediator

Design problem in TIVIDY:

Workflow objects, state objects, policies, must collaborate without every object depending directly on every other object.

GoF participant mapping:

- Mediator: WorkflowMediator

- Concrete mediator: none

- Colleagues: AbstractWorkItem and the other registered workflow colleagues

How participants collaborate:

Colleagues communicate workflow-relevant changes to the mediator. ConcreteWorkflowMediator coordinates the response: it applies its Strategy, coordinates state-related behaviour, and notifies registered observers. The colleague objects should not coordinate each other directly.

What may vary:

- Which colleagues participate in a workflow

- The coordination rules applied after a state change or workflow event

Why this pattern is appropriate:

TIVIDY is a coordination engine. Mediator centralises collaboration rules and avoids a network of direct

dependencies between the reusable workflow components.

Chain of Responsibility

Design problem in TIVIDY:

An escalation request may need to pass through a sequence of eligible handlers until one can deal with it.

GoF participant mapping:

- Handler: TaskEscalationManager

- Concrete handlers: ConcreteHandlerA, ConcreteHandlerB, ConcreteHandlerC, ConcreteHandlerD, ConcreteHandlerE

How participants collaborate:

The client submits a request to TaskEscalationManager through handleRequest(). A handler processes it when appropriate; otherwise it forwards the request to successor.

What may vary:

- The handlers included in an escalation path

- Their order and the conditions under which each handles a request

Why this pattern is appropriate:

The source of a workflow event should not need to know which participant will ultimately resolve it. Handlers can

be added or reordered without changing the requesting code.

Facade

Design problem in TIVIDY:

Clients and components need to interact with a complex network of workflow management operations (such as creating workItems, submitting work, managing pause/resume operations, tracking states, and handling dependencies) without having to manage all the internal subsystem interactions directly.

GoF participant mapping:

- Facade: TIVIDY

- Subsystem classes**:** WorkflowMediator, Creator, PrototypeManager, TaskEscalationManager, WorkItem

How participants collaborate:

The client communicates exclusively with the TIVIDY class, calling simplified high-level functions like createWorkItem(), submitWork(), pauseWork(), or printStatus(). TIVIDY then delegates these requests to the appropriate subsystem classes (such as WorkflowMediator and Creator) behind the scenes.

What may vary:

- The internal workflows, mediator routes, and object instantiation mechanisms used to process a request

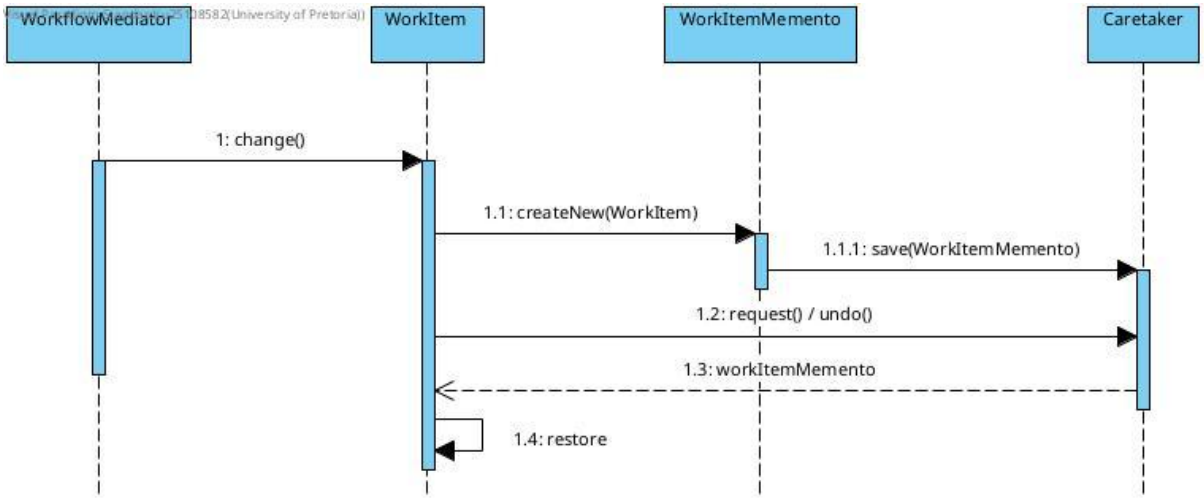
- The specific subsystem components involved in completing a high-level command

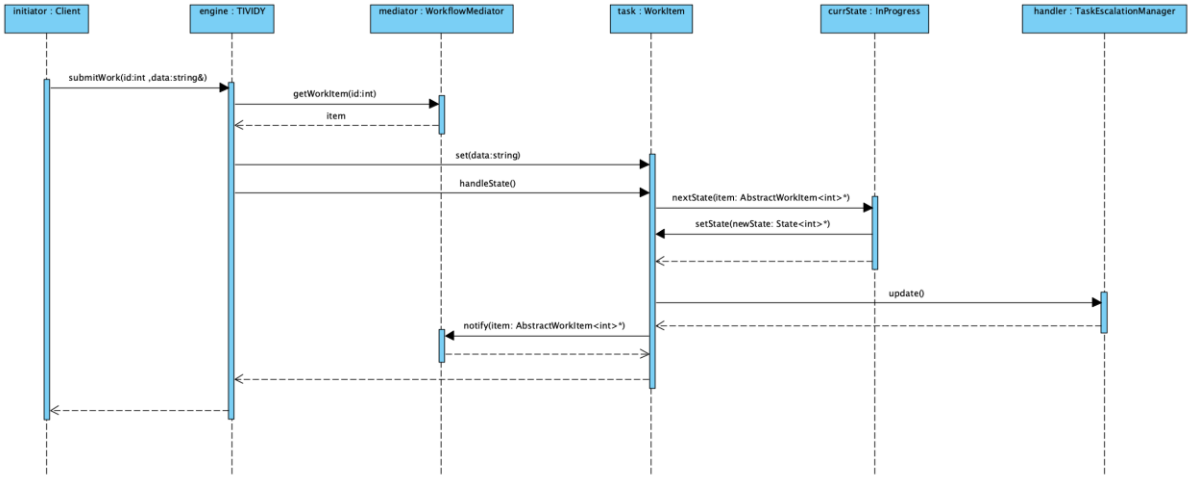
Why this pattern is appropriate:

Facade provides a clean, single entry point to the TIVIDY engine, reducing coupling between client code and internal workflow subsystems while making the system easier to use and maintain.

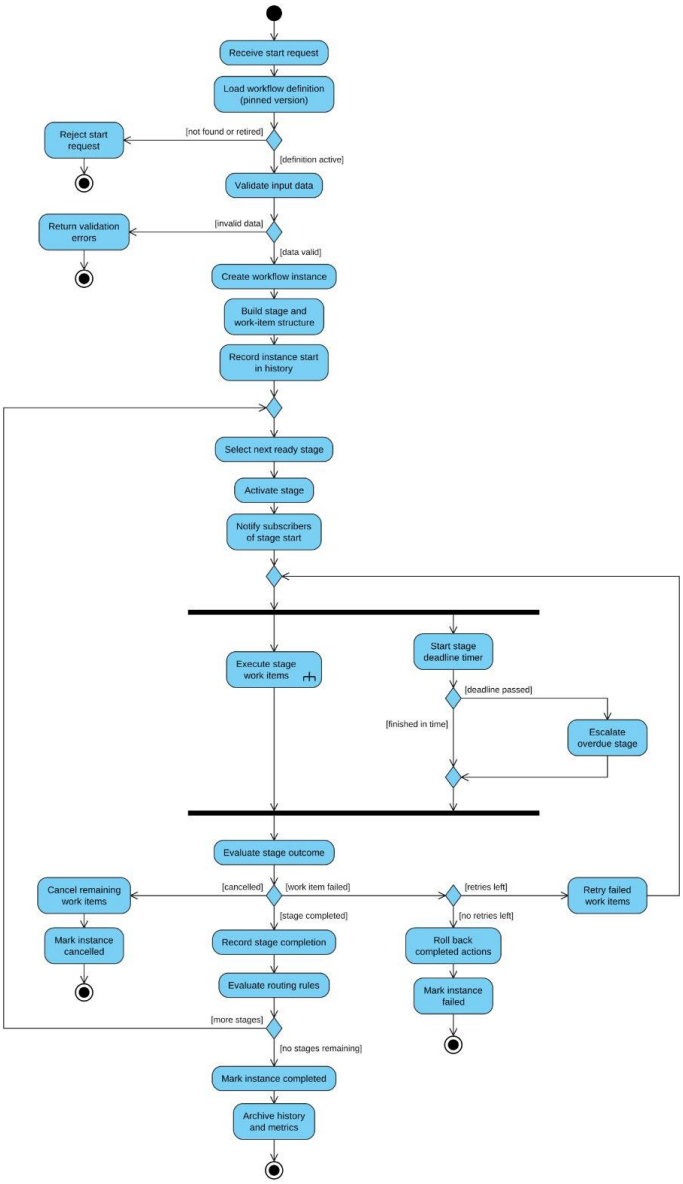
Other Diagrams

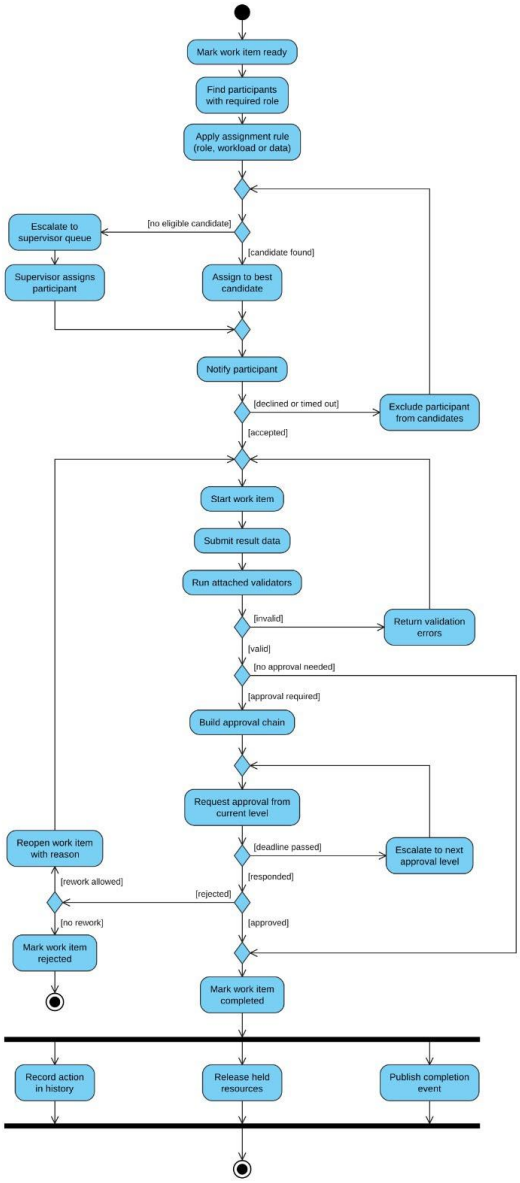
Sequence diagrams

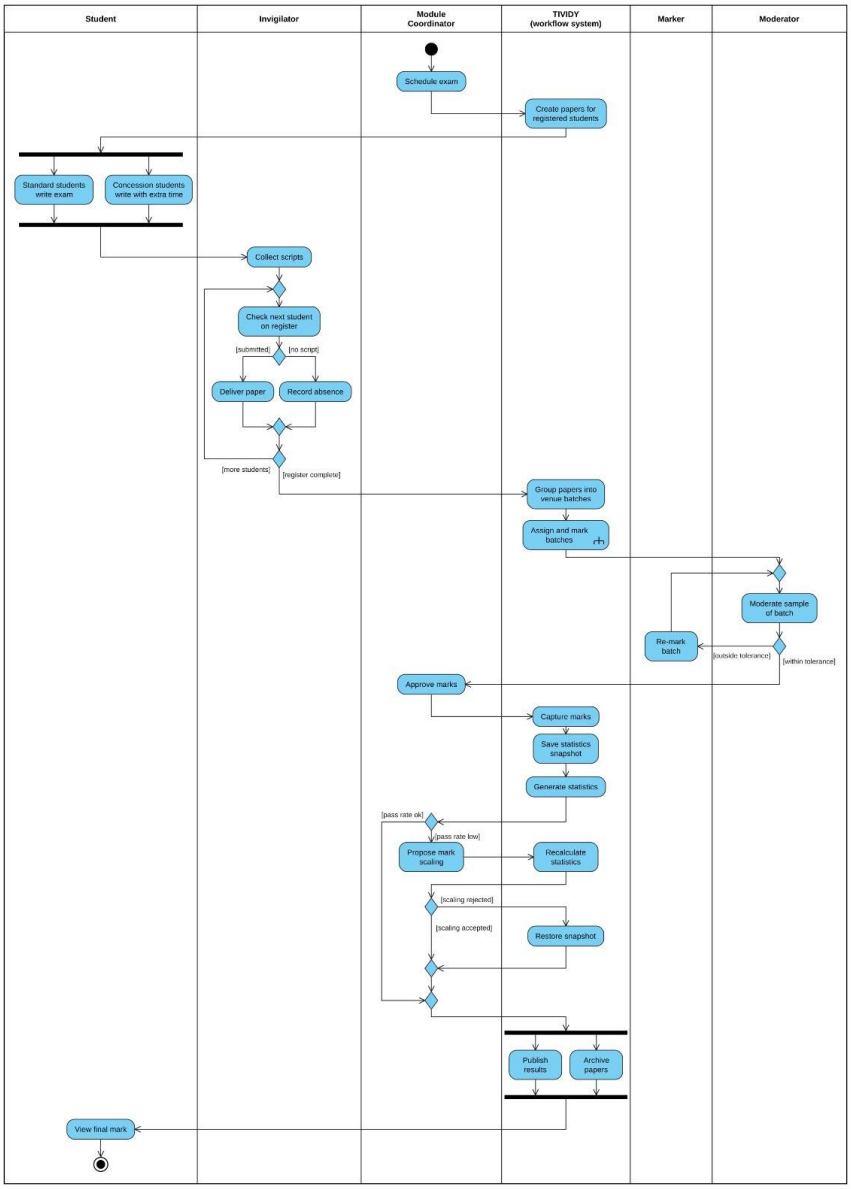




Activity diagrams






Student
Invigilator
Module Coordinator
TIVIDY (workflow system)
Marker
Moderator
Schedule exam
Create papers for registered students
Standard students write exam
Concession students write with extra time
Collect scripts
Check next student on register
[submitted]
[no script]
Deliver paper
Record absence
[more students]
[register complete]
Group papers into venue batches
Assign and mark batches
Moderate sample of batch
Re-mark batch
[outside tolerance]
[within tolerance]
Approve marks
Capture marks
Save statistics snapshot
Generate statistics
[pass rate ok]
[pass rate low]
Propose mark scaling
Recalculate statistics
[scaling rejected]
[scaling accepted]
Restore snapshot
Publish results
Archive papers
View final mark

Implementation explanation

Our initial implementation of the TIVIDY engine was very bare and basic with minimal functionality. This was done so that we could test if all our classes work together with minimal consequences if they did not as it would be an easy fix.

Once we had the bare basics working together, we started adding more functionality. We tested our code often to ensure all our new code is working together well enough. When we had a solid structure, we started adding a bunch of extra features that would be available for use by industries if needed.

From our initial design we redesigned our system to be fully generic and template based. By removing domain-specific hardcoding, TIVIDY can dynamically simulate workflows across any tech-enabled industry.

With all our new features and our engine being templatized we have a system that is very accommodating towards specifics that may want to use it.

Design Decision History

Revision 1: Composite Pattern Structure Alignment

- **Problem Considered :** The UML diagram modeled WorkItemComponent as an independent base class, causing CompositeWorkItem to inherit from WorkItemComponent rather than AbstractWorkItem. Additionally, CompositeWorkItem stored a child list typed strictly to CompositeWorkItem.

- **Original Design/Assumption :** CompositeWorkItem relied on a separate WorkItemComponent hierarchy, and its internal container could only hold other composite nodes.

- **Change Made :** Removed WorkItemComponent. Made CompositeWorkItem inherit directly from AbstractWorkItem. Updated child list storage in CompositeWorkItem to List<AbstractWorkItem*>.

- **Why change was made :** To strictly comply with the GoF Composite Pattern. Inheriting from AbstractWorkItem allows CompositeWorkItem to act as a standard Component. Typing the child list to AbstractWorkItem* enables recursive composition of leaf nodes (WorkItem), sub-composites, and decorated items.

Revision 2: Open/Closed Principle (OCP) in WorkItemState

- **Problem Considered :** Concrete states declared transition action methods (such as paused(), retry(), failedDelivery(), exception()) that were missing from the abstract base class WorkItemState.

- **Original Design/Assumption :** WorkItemState declared only nextState(), forcing callers or WorkItem to know concrete state types or rely on non-polymorphic routing.

- **Change Made :** Added virtual method signatures for all state transition actions (paused(), resume(), retry(), failed(), exception(), validate(), operation()) to the base WorkItemState class.

- **Why change was made :** To eliminate Open/Closed Principle (OCP) violations and prevent the need for type checks, dynamic casting, or switch/if-else chains when invoking state transitions polymorphically through a WorkItemState* pointer.

Revision 3: Observer Responsibility Decoupling from Mediator

- **Problem Considered :** WorkflowMediator contained observer list management functionality (-observerList: Observer*, attach(), detach(), notify()).

- **Original Design/Assumption :** The mediator acted as both a colleague coordinator and an observer subscriber registry.

- **Change Made :** Removed observer list attributes and attach()/detach() methods from WorkflowMediator. AbstractWorkItem remains the sole Subject base class managing subscriber lists (-observers: List<Observer*>).

- **Why change was made :** To preserve single-responsibility separation between the Mediator pattern (multi-colleague orchestration) and the Observer pattern (state-change notifications).

**** [Revision 4: Removing Strategy from TIVIDY**

- **Problem Considered :** Identifying algorithms applicable across various industries.

- **Original Design/Assumption :** Assumed algorithm classes could simply output a function call trace without concrete functionality.

- **Change Made :** Removed the Strategy design pattern from the architecture.

- **Why change was made :** The pattern could not provide generic functionality useful across diverse industry workflows.**]****

For this revision we ended up consulting with our mentor on it further and from there we had figured how we should be able to implement Strategy into back into TIVIDY.

Revision 5: Scenarios become workflow definitions run by the generic engine

- **Problem Considered :** The exam and game scenarios were built as separate class hierarchies (ExamPaper, Marker, GameWorker, their own states, strategies and facades) beside TIVIDY. The scenario was defining its own architecture, which the brief forbids: "Your selected scenario must not determine the complete architecture of the generic workflow manager", and our research found "all process rules live in the definition, not the engine" (WfMC, 1996).

- **Original Design/Assumption :** Each scenario needed its own classes and its own UI. The generic engine hard-coded its escalation chain (Team lead -> Manager -> Department head -> Director -> Executive) and its external system.

- **Change Made :** The escalation chain (1 to 5 named levels with rising limits), the external system's name and the scoring rule are now set at run time through the facade (setEscalationLevels, setExternalSystem, setStrategy). Added scoreTask and named template copies so instances can be marked and told apart. The web UI loads a workflow definition (exam marking, game studio, software project, or one the user builds) that also names the work, its parts, its states and its extra behaviours. The scenario code and pages stay in the repository but are no longer built or served.

- **Why change was made :** So, one engine runs any process without being rewritten, as the brief and our research require. Exam marking is now Tutor ->Assistant lecturer -> Lecturer -> Module coordinator escalation, a master paper template copied per student (Prototype), questions as scored tasks (Strategy), concessions as Decorators and a semester test as a Composite, all on the generic classes.

Revision 6: Added a vector containing structs in AbstractWorkItem

- **Problem Considered :** How Would we be able to access the various tasks that a WorkItem may contain. We were thinking about our scenarios and how we would have a WorkItem contain exam questions and maybe levels for a game.

- **Original Design/Assumption :** We were initially planning on just having variables however we felt as if that was not generic enough to accommodate what we wanted our system to be able to do. However, with much thought we could not even find a way to initialise variables to fit both of our scenarios let alone multiple more.

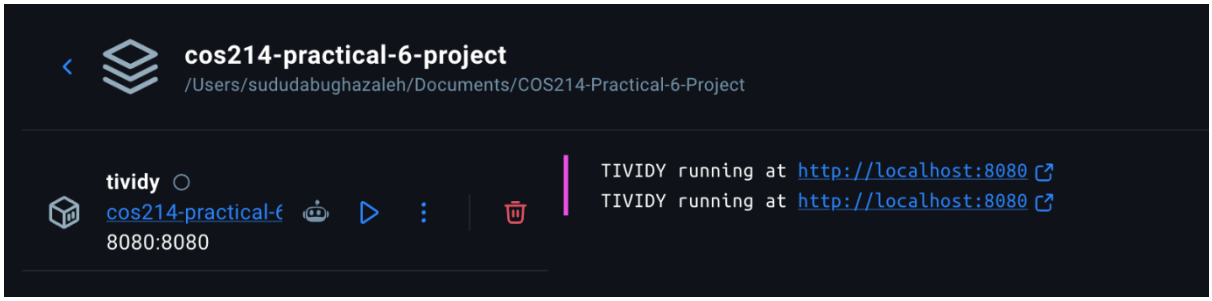
- **Change Made :** We added a struct called TaskRecords (the struct contains a name with type string and a score and maxScore of type int) and had a vector that contains TaskRecords called tasks in AbstractWorkItem. However,

we needed a quick and easy way to modify the tasks vector thus we added some functions that allow us to add a task, get a task or all the tasks, get the count of how many tasks there are and to update a task score.

- **Why change was made :** The change was made to also allow us to calculate algorithms from Startegy based on the AbstractWorkItem. We were able to calculate our minimum and maximum scores for example with greater ease. It also allowed us to have an easier implementation for Chain of Responsibility as we would pass to the next handler based on how many tasks were done.

Engineering evidence

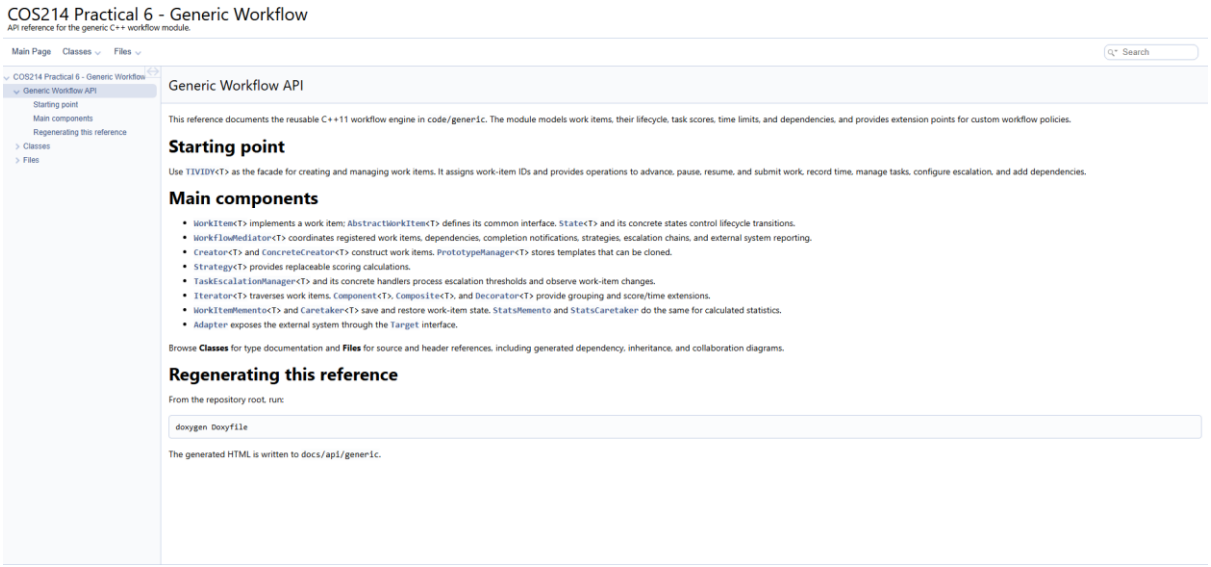
Using Docker to Build and Run the system




```
[+] Running 2/2
 ✓ tividy                                    Built
 ✓ Container cos214-practical-6-project-tividy-1  Recreated
Attaching to tividy-1
tividy-1  | TIVIDY running at http://localhost:8080
```

```
tamly@BumbleBee:~/cos214/COS214_PA6/COS214-Practical-6-Project$ docker compose up --build
WARN[0001] Docker Compose is configured to build using Bake, but buildx isn't installed
[+] Building 1.3s (16/16) FINISHED                                                                                    docker:default
 => [tividy internal] load build definition from Dockerfile                                                                    0.1s
 => => transferring dockerfile: 433B                                                                                           0.1s
 => [tividy internal] load metadata for docker.io/library/debian:bookworm-slim                                                 0.1s
 => [tividy internal] load metadata for docker.io/library/gcc:13.2.0                                                           0.1s
 => [tividy internal] load .dockerignore                                                                                       0.0s
 => => transferring context: 50B                                                                                               0.0s
 => [tividy internal] load build context                                                                                       0.1s
 => => transferring context: 3.63kB                                                                                            0.1s
 => [tividy build 1/5] FROM docker.io/library/gcc:13.2.0@sha256:15c73bc59ae88b3fd563ef2ec4a8743a8848a9f74362b6d116c4543c4844b6e0  0.1s
 => => resolve docker.io/library/gcc:13.2.0@sha256:15c73bc59ae88b3fd563ef2ec4a8743a8848a9f74362b6d116c4543c4844b6e0              0.1s
 => [tividy stage-1 1/4] FROM docker.io/library/debian:bookworm-slim@sha256:99c0b25c46d0214d5a2091092e7908907dcaecd6a962cde0ac700acee222b6f3  0.1s
 => => resolve docker.io/library/debian:bookworm-slim@sha256:99c0b25c46d0214d5a2091092e7908907dcaecd6a962cde0ac700acee222b6f3  0.1s
 => CACHED [tividy stage-1 2/4] WORKDIR /app                                                                                   0.0s
 => CACHED [tividy build 2/5] WORKDIR /src                                                                                     0.0s
 => CACHED [tividy build 3/5] COPY server/ server/                                                                             0.0s
 => CACHED [tividy build 4/5] COPY code/generic/ code/generic/                                                                 0.0s
 => CACHED [tividy build 5/5] RUN g++ -std=c++11 -O2 -pthread -static-libstdc++ -static-libgcc -Icode/generic -Iserver     server/*.cpp $(find code/gene  0.0s
 => CACHED [tividy stage-1 3/4] COPY --from=build /src/tividy .                                                                0.0s
 => CACHED [tividy stage-1 4/4] COPY web/generic/ web/generic/                                                                 0.0s
 => [tividy] exporting to image                                                                                                0.4s
 => => exporting layers                                                                                                        0.0s
 => => exporting manifest sha256:0b7dbd519f092bdb809d0aff96dd67b96f1cd3a5d19765dfa4e300f2b6695a1b                              0.0s
 => => exporting config sha256:e32c530ced1137d79b1f77172346a079a3f3be84c544e6204052f14802e86221                                0.0s
 => => exporting attestation manifest sha256:00100b8de4088f91ac5afb1e61e1c2bcd4f3fd7f0794a2bec5b73a4bfadb0291                  0.1s
 => => exporting manifest list sha256:3a638422bb9fecec86f01421fe949dce51748027b34fb6de721d25fcbc935ce0                         0.0s
 => => naming to docker.io/library/cos214-practical-6-project-tividy:latest                                                    0.0s
 => => unpacking to docker.io/library/cos214-practical-6-project-tividy:latest                                                 0.0s
 => [tividy] resolving provenance for metadata file                                                                            0.0s
[+] Running 2/2
 ✓ tividy                                    Built                                                                             0.0s
 ✓ Container cos214-practical-6-project-tividy-1  Recreated                                                                   11.6s
```

Using Doxygen for documentation




```cpp
/** @brief Score and maximum score for a named task within a work item. */
struct TaskRecord {
    std::string name;
    int score;
    int maxScore;
    bool scored;

    /** @brief Creates an empty task record. */
    TaskRecord() : name(""), score(0), maxScore(0), scored(false) {}
    /** @brief Creates a task record with the supplied score values.
     *  @param n Task name.
     *  @param s Current task score.
     *  @param ms Maximum task score.
     */
    TaskRecord(const std::string& n, int s, int ms) : name(n), score(s), maxScore(ms), scored(true) {}
    /** @brief Creates a task record that has not been scored yet.
     *  @param n Task name.
     *  @param ms Maximum task score.
     */
    TaskRecord(const std::string& n, int ms) : name(n), score(0), maxScore(ms), scored(false) {}
};
```

```cpp
/** @brief Base decorator that forwards component operations. */
template <typename T>
class Decorator : public Component<T> {
public:

    /** @brief Wraps a component with decorator behavior.
     *  @param component Component being decorated.
     */
    Decorator(Component<T>* component);
    /** @brief Displays the wrapped component.
     *  @param indent Indentation level used for hierarchical output.
     */
    void display(int indent = 0);
    /** @brief Destroys the decorator. */
    virtual ~Decorator();

    /** @brief Gets the wrapped component's score. */
    int getScore();
    /** @brief Gets the wrapped component's time allowance. */
    int getTime();

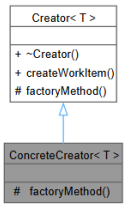
protected:
    Component<T>* component ;//component to deco
};
```

ConcreteCreator< T > Class Template Reference

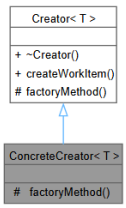
Creates concrete **WorkItem** instances through the **Creator** interface. More...

`#include <ConcreteCreator.h>`

Inheritance diagram for ConcreteCreator< T >:



Collaboration diagram for ConcreteCreator< T >:



Protected Member Functions

`AbstractWorkItem< T > * factoryMethod (int id, const std::string &name, int deadline, WorkflowMediator< T > *mediator)`	
	Constructs a concrete work item.

Additional Inherited Members

Public Member Functions inherited from **Creator< T >**

`virtual ~Creator ()`	
	Destroys the creator through its base interface.
`AbstractWorkItem< T > * createWorkItem (int id, const std::string &name, int deadline, WorkflowMediator< T > *mediator)`	
	Creates a work item using the concrete factory method.

Detailed Description

template<typename T>
class ConcreteCreator< T >

Creates concrete **WorkItem** instances through the **Creator** interface.

Definition at line **8** of file **ConcreteCreator.h**.

Member Function Documentation

◆ factoryMethod()

```
template<typename T>
AbstractWorkItem< T > * ConcreteCreator< T >::factoryMethod ( int                  id,
                                                              const std::string &  name,
                                                              int                  deadline,
                                                              WorkflowMediator< T > * mediator )
```

protected virtual

Constructs a concrete work item.

Parameters

id	Work-item identifier.
name	Work-item name.
deadline	Allowed work duration.
mediator	Mediator coordinating the work item.

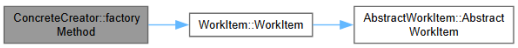
Returns

The newly created work item.

Implements **Creator< T >**.

Definition at line **8** of file **ConcreteCreator.cpp**.

Here is the call graph for this function:



The documentation for this class was generated from the following files:

- code/generic/**ConcreteCreator.h**
- code/generic/**ConcreteCreator.cpp**

Testing evidence

The following commands were used to run tests :

- ctest --test-dir build --output-on-failure.

  This is for the broadest proof of stuff passing.

- .\build\workflow_tests.exe.

  Captures the test list and final [PASSED] count.

- .\build\workflow_tests.exe --gtest_filter="StateTest.*:MediatorTest.*"

  Shows selected generic test results

Testing examples

```
26/34 Test #26: IteratorTest.VisitsEveryWorkItemInOrderThenStops ..........................   Passed    0.05 sec
      Start 27: IteratorTest.EmptyMediatorHasNothingToVisit
27/34 Test #27: IteratorTest.EmptyMediatorHasNothingToVisit ...............................   Passed    0.05 sec
      Start 28: EscalationTest.OnTimeWorkIsObservedButNotEscalated
28/34 Test #28: EscalationTest.OnTimeWorkIsObservedButNotEscalated ........................   Passed    0.04 sec
      Start 29: EscalationTest.OverdueWorkGoesFurtherUpTheChainTheLaterItIs
29/34 Test #29: EscalationTest.OverdueWorkGoesFurtherUpTheChainTheLaterItIs ...............   Passed    0.04 sec
      Start 30: CompletionTest.MediatorScoresNotifiesExternalSystemAndDeliversDependents
30/34 Test #30: CompletionTest.MediatorScoresNotifiesExternalSystemAndDeliversDependents ...   Passed    0.04 sec
      Start 31: CompletionTest.SwappingTheStrategyChangesHowCompletedWorkIsScored
31/34 Test #31: CompletionTest.SwappingTheStrategyChangesHowCompletedWorkIsScored ..........   Passed    0.04 sec
      Start 32: PrototypeTest.TemplateCloneIsANewIndependentItem
32/34 Test #32: PrototypeTest.TemplateCloneIsANewIndependentItem ...........................   Passed    0.04 sec
      Start 33: PrototypeTest.UnknownTemplatesAndItemsAreRejected
33/34 Test #33: PrototypeTest.UnknownTemplatesAndItemsAreRejected ..........................   Passed    0.04 sec
      Start 34: PrototypeTest.ManagerOwnsAndRemovesItsPrototypes
34/34 Test #34: PrototypeTest.ManagerOwnsAndRemovesItsPrototypes ...........................   Passed    0.04 sec

100% tests passed out of 34

Total Test time (real) =   1.56 sec
```

```
[----------] 3 tests from PrototypeTest
[ RUN      ] PrototypeTest.TemplateCloneIsANewIndependentItem
  [Factory] Creating WorkItem 'Design'
  [TIVIDY] Created 'Design' (id 1)
[WorkItem] Added task: Wireframes (6/10)
  [State] Design: Created -> Delivered
Team lead was notified that Design is Delivered
  [State] Design: Delivered -> InProgress
Team lead was notified that Design is InProgress
  [State] Design: InProgress -> Validation
Team lead was notified that Design is Validation
  [State] Design: Validation -> Completed
Team lead was notified that Design is Completed
[Strategy A] Calculated overall average score: 6
  [Mediator] Telling the external system that Design is complete
(Adapter) Sending request
(Adaptee) Specified request recieved
  [TIVIDY] Saved 'Design' as template 0
  [TIVIDY] Created 'Design' (id 2) from template 0
  [TIVIDY] Created 'Design' (id 3) from template 0
[WorkItem] Updated: Wireframes score to 1
[       OK ] PrototypeTest.TemplateCloneIsANewIndependentItem (21 ms)
[ RUN      ] PrototypeTest.UnknownTemplatesAndItemsAreRejected
  [TIVIDY] No work item with id 42
  [TIVIDY] No template 0
  [TIVIDY] No template -1
[       OK ] PrototypeTest.UnknownTemplatesAndItemsAreRejected (2 ms)
[ RUN      ] PrototypeTest.ManagerOwnsAndRemovesItsPrototypes
[       OK ] PrototypeTest.ManagerOwnsAndRemovesItsPrototypes (0 ms)
[----------] 3 tests from PrototypeTest (46 ms total)

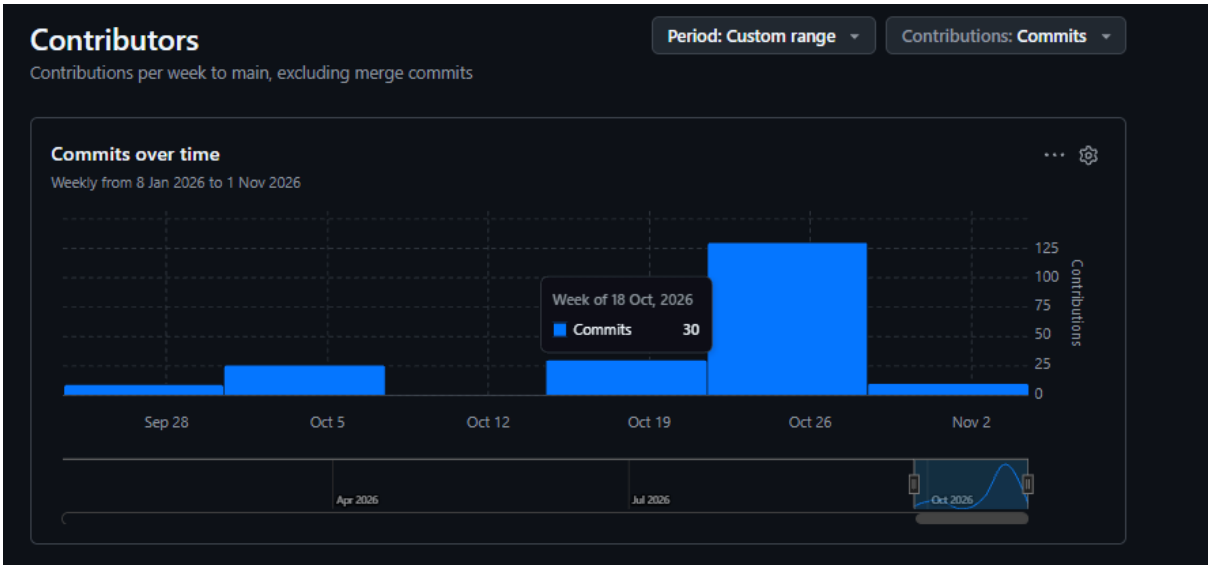
[----------] Global test environment tear-down
[==========] 34 tests from 14 test suites ran. (502 ms total)
[  PASSED  ] 34 tests.
```

```
C:\Users\YKwagga\Documents\GitHub\COS214-Practical-6-Project>.\build\workflow_tests.exe
Running main() from C:\Users\YKwagga\Documents\GitHub\COS214-Practical-6-Project\build\_deps\googletest-src\googletest\src\gtest_main.cc
[==========] Running 34 tests from 14 test suites.
[----------] Global test environment set-up.
[----------] 3 tests from StateTest
[ RUN      ] StateTest.ValidDataCompletesAndInvalidDataReturnsToInProgress
  [State] Task: Created -> Delivered
  [State] Task: Delivered -> InProgress
  [State] Task: InProgress -> Validation
  [State] Task: Validation -> InProgress
  [State] Task: InProgress -> Validation
  [State] Task: Validation -> Completed
[       OK ] StateTest.ValidDataCompletesAndInvalidDataReturnsToInProgress (2 ms)
[ RUN      ] StateTest.DeadlineGuardPreventsValidation
  [State] Late task: Created -> Delivered
  [State] Late task: Delivered -> InProgress
  [State] guard [withinDeadline] failed for Late task
[       OK ] StateTest.DeadlineGuardPreventsValidation (1 ms)
[ RUN      ] StateTest.PauseAndResumeRestoreInProgress
  [State] Task: Created -> Delivered
  [State] Task: Delivered -> InProgress
  [State] Task: InProgress -> Paused
  [State] Task: Paused -> InProgress
[       OK ] StateTest.PauseAndResumeRestoreInProgress (2 ms)
[----------] 3 tests from StateTest (21 ms total)
```

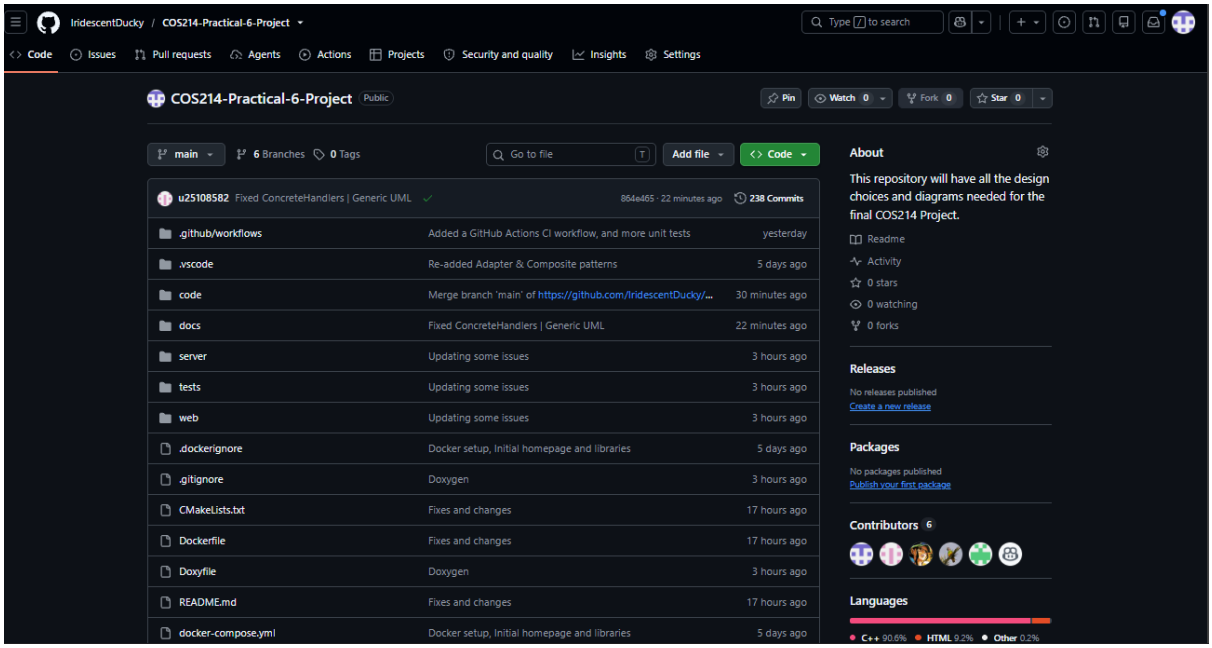
```
C:\Users\YKwagga\Documents\GitHub\COS214-Practical-6-Project>ctest --test-dir build --output-on-failure
Test project C:/Users/YKwagga/Documents/GitHub/COS214-Practical-6-Project/build
      Start  1: StateTest.ValidDataCompletesAndInvalidDataReturnsToInProgress
 1/34 Test  #1: StateTest.ValidDataCompletesAndInvalidDataReturnsToInProgress .....   Passed    0.04 sec
      Start  2: StateTest.DeadlineGuardPreventsValidation
 2/34 Test  #2: StateTest.DeadlineGuardPreventsValidation ...............................   Passed    0.04 sec
      Start  3: StateTest.PauseAndResumeRestoreInProgress
 3/34 Test  #3: StateTest.PauseAndResumeRestoreInProgress ...............................   Passed    0.05 sec
      Start  4: MediatorTest.RejectsInvalidAndDuplicateDependencies
 4/34 Test  #4: MediatorTest.RejectsInvalidAndDuplicateDependencies .....................   Passed    0.04 sec
      Start  5: MediatorTest.CompletionActivatesEveryCreatedDependentOnlyOnce
 5/34 Test  #5: MediatorTest.CompletionActivatesEveryCreatedDependentOnlyOnce .....   Passed    0.05 sec
      Start  6: MediatorTest.IncompletePredecessorDoesNotActivateDependents
 6/34 Test  #6: MediatorTest.IncompletePredecessorDoesNotActivateDependents .....   Passed    0.04 sec
      Start  7: FacadeTest.WorkflowsKeepItemsAndDependenciesInstanceLocal
 7/34 Test  #7: FacadeTest.WorkflowsKeepItemsAndDependenciesInstanceLocal .....   Passed    0.05 sec
      Start  8: FacadeTest.NullOrUnknownIdsAreIgnoredSafely
 8/34 Test  #8: FacadeTest.NullOrUnknownIdsAreIgnoredSafely .....   Passed    0.04 sec
      Start  9: FacadeTest.InvalidTimeAndTasksAreRejected
 9/34 Test  #9: FacadeTest.InvalidTimeAndTasksAreRejected .....   Passed    0.04 sec
      Start 10: MementoTest.RestoresAnIndependentStateSnapshot
10/34 Test #10: MementoTest.RestoresAnIndependentStateSnapshot .....   Passed    0.04 sec
      Start 11: ObserverTest.AttachedObserverIsNotifiedAndDetachedObserverIsNot
11/34 Test #11: ObserverTest.AttachedObserverIsNotifiedAndDetachedObserverIsNot .....   Passed    0.04 sec
      Start 12: ObserverTest.AttachingTwiceStillNotifiesOnce
12/34 Test #12: ObserverTest.AttachingTwiceStillNotifiesOnce .....   Passed    0.04 sec
      Start 13: ObserverTest.AttachingToAnotherItemMovesTheObserver
13/34 Test #13: ObserverTest.AttachingToAnotherItemMovesTheObserver .....   Passed    0.04 sec
      Start 14: ObserverTest.DestroyedObserverDetachesItself
14/34 Test #14: ObserverTest.DestroyedObserverDetachesItself .....   Passed    0.05 sec
      Start 15: ObserverTest.DestroyedWorkItemReleasesItsObservers
15/34 Test #15: ObserverTest.DestroyedWorkItemReleasesItsObservers .....   Passed    0.04 sec
      Start 16: DecoratorTest.BonusesStackOnTheWrappedWorkItem
16/34 Test #16: DecoratorTest.BonusesStackOnTheWrappedWorkItem .....   Passed    0.04 sec
      Start 17: CompositeTest.GroupAddsUpItsChildren
17/34 Test #17: CompositeTest.GroupAddsUpItsChildren .....   Passed    0.04 sec
      Start 18: CompositeTest.GroupDoesNotDeleteWorkItemsOwnedByTheFacade
18/34 Test #18: CompositeTest.GroupDoesNotDeleteWorkItemsOwnedByTheFacade .....   Passed    0.04 sec
      Start 19: FactoryMethodTest.CreatorMakesTheRequestedWorkItem
19/34 Test #19: FactoryMethodTest.CreatorMakesTheRequestedWorkItem .....   Passed    0.04 sec
      Start 20: FactoryMethodTest.FacadeCreatesItemsThroughTheFactory
20/34 Test #20: FactoryMethodTest.FacadeCreatesItemsThroughTheFactory .....   Passed    0.04 sec
      Start 21: ChainOfResponsibilityTest.ReplacingASuccessorUsesTheNewOne
21/34 Test #21: ChainOfResponsibilityTest.ReplacingASuccessorUsesTheNewOne .....   Passed    0.04 sec
      Start 22: ChainOfResponsibilityTest.SuccessorThatWouldLoopIsRejected
22/34 Test #22: ChainOfResponsibilityTest.SuccessorThatWouldLoopIsRejected .....   Passed    0.05 sec
      Start 23: ChainOfResponsibilityTest.RequestGoesToFirstHandlerThatCanTakeIt
23/34 Test #23: ChainOfResponsibilityTest.RequestGoesToFirstHandlerThatCanTakeIt .....   Passed    0.04 sec
      Start 24: ChainOfResponsibilityTest.FiveHandlerTypesCanFormOneChain
24/34 Test #24: ChainOfResponsibilityTest.FiveHandlerTypesCanFormOneChain .....   Passed    0.04 sec
      Start 25: WorkflowEndToEndTest.FacadeCompletesPredecessorAndActivatesDependent
25/34 Test #25: WorkflowEndToEndTest.FacadeCompletesPredecessorAndActivatesDependent .....   Passed    0.05 sec
```

GitHub evidence

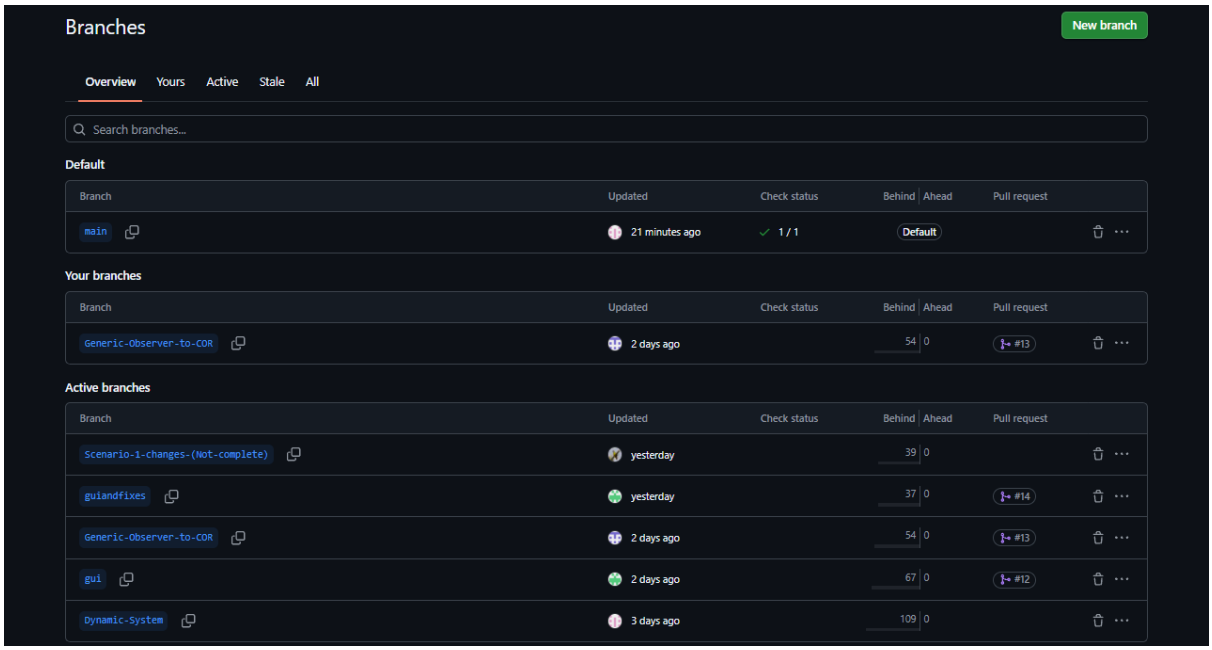
Our Contributions



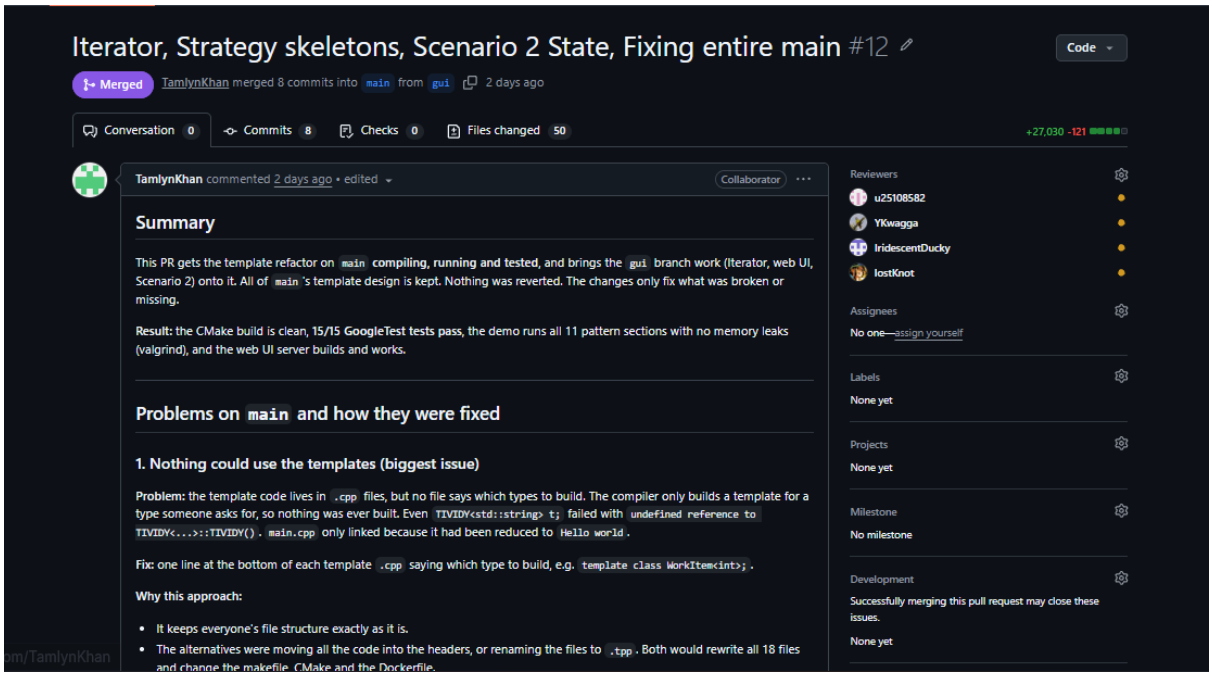


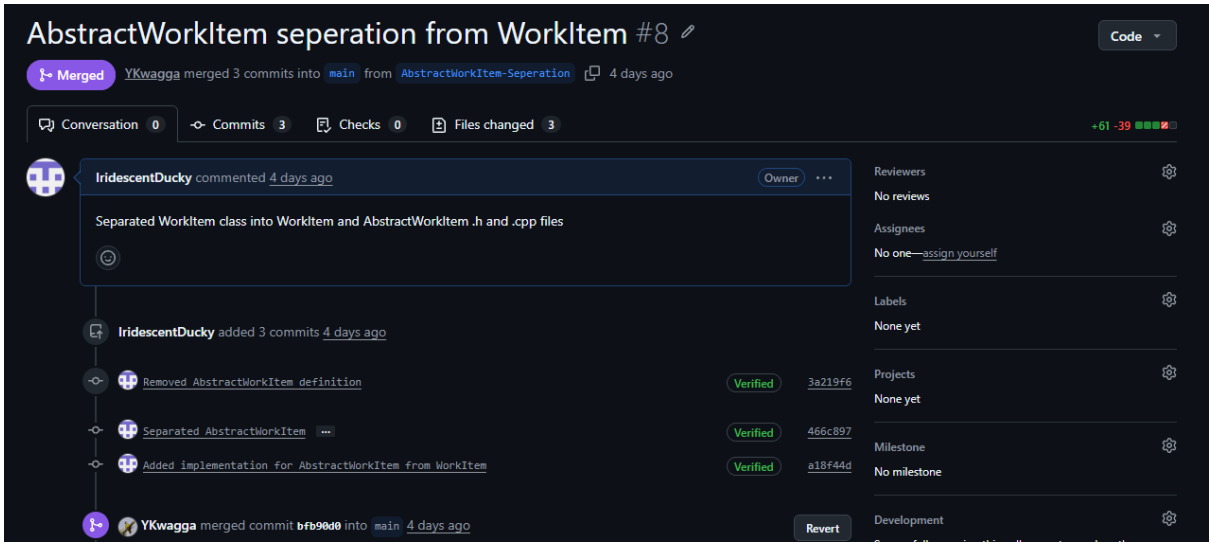


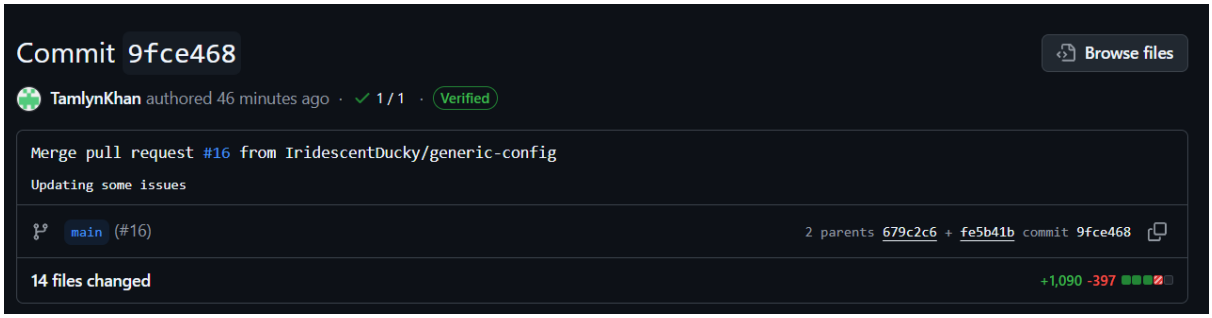
Branches Example



Pull Request, Merges and Commits








Commit a2ef853
Browse files
u25108582 committed 31 minutes ago · 1 / 1
Merge branch 'main' of https://github.com/IridescentDucky/COS214-Practical-6-Project
main
2 parents 36fd9cd + 9fce468 commit a2ef853
14 files changed
+1,090 -397


Commit 84d705e
Browse files
TamlynKhan authored 2 weeks ago
Verified
Merge pull request #2 from IridescentDucky/Tamlyn-(Facade,-State)
Tamlyn (facade, state)
main (#2)
2 parents b6ed59e + db57078 commit 84d705e
10 files changed
+753 -1